

Laboratorio 3 - Sviluppo di un semplice videogioco 2D

In questa esercitazione svilupperemo da zero un semplice videogioco ispirato a Breakout [Atari, 1976] e Arkanoid [Taito, 1986].

I giochi di questo tipo hanno tre elementi fondamentali: una racchetta, controllata dal giocatore; una pallina, che si muove autonomamente e rimbalza all'infinito dentro al campo di gioco, e un muro di mattoncini. La racchetta si muove lungo il bordo inferiore della finestra, e il giocatore deve impedire alla pallina di cadere nel bordo inferiore, che è l'unico aperto. La pallina rimbalza contro i bordi laterali e il bordo superiore, e contro i mattoncini del muro. Quando un mattoncino viene colpito, si rompe e scompare. Il gioco finisce quando il giocatore distrugge tutti i mattoncini.

L'esercitazione sarà resa disponibile in tappe incrementali; per ispezionare cosa è cambiato esattamente fra una tappa e l'altra, potete usare `git diff` da terminale:

```
git diff --no-index -- previous_chapter.cpp next_chapter.cpp
```

Introduzione - template di partenza

Partire dal file `blockbreaker.cpp` nella cartella `blockbreaker-template`. Il codice apre una finestra 800x600, definisce le callback per gestirne la chiusura ed il resize con proporzione bloccata e ascolta gli eventi in loop.

Riferimenti:

pagine [window](#) ed [events](#) nei tutorial SFML.

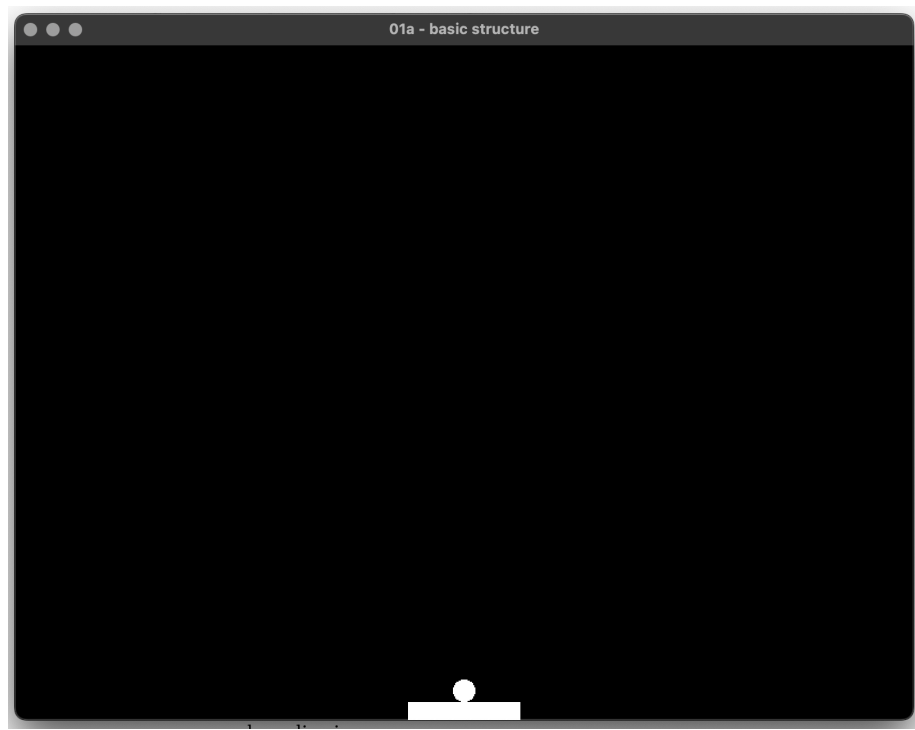
Capitolo 1 - Struttura del gioco, palla e racchetta

Strutture dati fondamentali. Creazione di un elemento racchetta e di un elemento palla, e loro visualizzazione a schermo.

Parte 1.a - Struttura e disegno

Obiettivo:

Creare una racchetta e una pallina nello stato interno del gioco, e disegnarne a schermo una rappresentazione schematica, come nella seguente figura:



Riferimenti:

Pagine [draw](#) e [shapes](#) nei tutorial SFML.

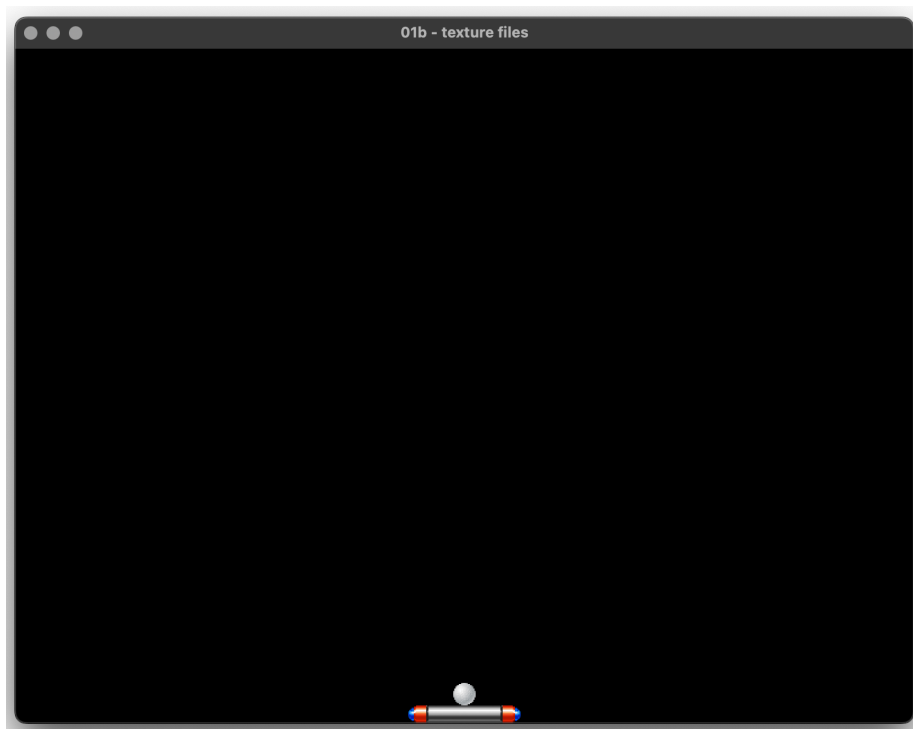
Sviluppo:

- Aggiungere due costanti globali `ball_radius = 10.0` e `paddle_size = {100.0, 16.0}` per fissare le dimensioni della pallina (da disegnare con una `CircleShape`) e racchetta (da disegnare con una `RectangleShape`). - Creare tre struct: una per **Paddle** la racchetta, una **Ball** per la pallina e una **State** per lo stato; **Paddle** e **Ball** hanno come campi la dimensione e posizione delle rispettive shape, mentre **State** contiene una racchetta e una pallina; tutte e tre le strutture forniscono un costruttore di default e un metodo `draw` che prende in input la finestra. - I costruttori di **Paddle** e **Ball** inizializzano la posizione e dimensione delle primitive, nella posizione mostrata nello screenshot (la racchetta in basso al centro, la pallina sopra la racchetta). - Il metodo `state.draw()` viene chiamato nel main loop, fra `window.clear()` e `window.display()`, e a sua volta chiama i metod `ball.draw()` e `paddle.draw()`. Ulteriore idea: per non tenere `window` come variabile globale, conviene passarla come parametro dei metodi `draw()`.

Parte 1.b - Texture da file

Obiettivo:

Racchetta e pallina vengono disegnate come nella seguente figura:



Le texture da usare sono contenute nei file `texture_ball.png` e `texture_paddle.png`.

Riferimenti: tutorial SFML [sprite and textures](#).

Sviluppo:

- Aggiungere i nomi dei file come costanti all'inizio del programma. Notare che il costruttore di `sf::Texture` prende in input un nome di file come stringa C, quindi bisogna dichiarare i nomi dei file come `const char* filename = "..."`, oppure nel caso si usi una stringa C++ dichiarata come `string filename`, bisogna poi passarla al costruttore della texture come `filename.c_str()`. - Se l'eseguibile compilato è nella cartella `build`, come di default, bisogna indicare i path col prefisso `../` per indicare la parent directory.

Problema:

cosa succede se si fa partire l'eseguibile da una cartella diversa da `build`, ad esempio lanciandolo dalla cartella di progetto invece che da dentro `build` (ovvero, `./build/exe` invece che `cd build; ./exe`?

Parte 1.c - Texture incorporate

Obiettivo:

Rendere il caricamento delle texture sempre funzionante, a prescindere da dove si trovano. Un possibile trucco consiste nell'incorporarle all'interno dell'eseguibile compilato. In questo modo il programma diventa autocontenuto, e si trova le

risorse già in memoria invece di doverle cercare nel filesystem e leggerle (anzi, l'eseguibile può essere spostato su un sistema su cui quelle immagini neanche ci sono).

Riferimenti:

Il [costruttore](#) di `Texture` che prende in input un array già presente in memoria e la sua lunghezza.

Sviluppo:

- Includere il file `textures.cpp` che contiene due array di byte (char) in cui sono contenute le due texture utilizzate in precedenza, codificate in esadecimale, e due variabili con la lunghezza di tali array. Il codice con gli array e la loro lunghezza è stato generato con il tool da linea di comando `xxd -i` (nel caso voleste inserire nuove texture o nuove risorse, è un comando che dovrebbe essere preinstallato in molte distribuzioni linux, su MacOS, e su Windows con WSL). - Le due stringhe che specificano i path dei file di texture ora non sono più necessarie. - Nei costruttori di `Ball` e `Paddle` si usa il costruttore appropriato di `sf::Texture`, visto che la texture ora è già in memoria, caricata con il resto dell'eseguibile e disponibile nei dati statici del programma.

Capitolo 2 - Movimento e interazione

Gestione del movimento della pallina e della racchetta. La pallina ha vita autonoma, mentre la racchetta è controllata dal giocatore.

Parte 2.a - Pallina

Obiettivo:

Muovere la pallina a una certa velocità e in una certa direzione alla partenza del gioco. La velocità e la traiettoria iniziali possono essere qualsiasi, ma avranno un valore iniziale di default.

Riferimenti:

- Le pagine [time](#) e [angles](#) nei tutorial SFML. - Il [costruttore in coordinate polari](#) della classe `sf::Vector2f`. - Nella documentazione SFML, guardare anche le informazioni relative agli eventi `KeyPressed`, `KeyReleased` e relativi scancode.

Esperimento iniziale:

- Provare a lanciare la pallina verso l'alto in direzione verticale, incrementando dentro al main loop `ball.pos.y`. Si può sperimentare con diversi valori di displacement incrementale. - Avendo il nostro gioco un framerate bloccato a 60 FPS (frame per seconds), i valori usati fino ad ora esprimono una velocità in (circa) pixel ogni sessantesimo di secondo: $\text{pixels}/(\text{seconds}/60)$. La cosa non è conveniente per diversi motivi, imprecisa, e non funziona per framerate variabili. - Bisogna usare un timer, ad esempio `sf::Clock`. Va creato nel `main` e a ogni giro del loop, subito dopo l'handling degli eventi, bisogna leggere quanto tempo è passato dal reset precedente e farlo ripartire. - Il displacement lungo le y della pallina va scalato rispetto al numero di secondi (o alla frazione di secondo)

intercorsa a ogni giro e misurata dal timer. In questo modo il displacement lungo le y diventa a tutti gli effetti la velocità della pallina, espressa in pixel al secondo. - Come si fa a muovere la pallina in una direzione qualsiasi, non solo in verticale? Ci vuole un displacement che lavori contemporaneamente lungo entrambi gli assi (`pos.x` e `pos.y`). Si può quindi aggiungere un displacement incrementale anche lungo le x. - Sperimentare cosa succede variando la due componenti della velocità. Se le due velocità sono uguali come si muove la pallina? Se invece prevale uno dei due assi ($v_x > v_y$, e viceversa)? - Modificare x e y separatamente non è particolarmente comodo: ogni volta che si modifica uno dei due valori, stiamo modificando contemporaneamente sia la velocità e la direzione della pallina. - Sarebbe meglio poter lavorare su velocità e direzione della pallina in modo indipendente. La classe `sf::Vector2f` ha un costruttore in coordinate polari che prende come parametri un modulo (scalare float) e un angolo (classe `sf::Angle`). Usare il costruttore per generare un vettore di displacement di `ball.pos`, e sperimentare cosa succede con diversi valori di modulo ed angolo. Facendo così, il modulo è la velocità in pixel/secondo lungo qualsiasi traiettoria, l'angolo è la direzione della traiettoria.

Sviluppo:

Ora ci sono tutti gli ingredienti fondamentali per gestire al meglio il movimento della pallina. Basta riorganizzare quanto scritto nella struttura del programma. - Aggiungere due variabili globali `ball_initial_speed` e `ball_initial_angle` per velocità e traiettoria di partenza della pallina. - Aggiungere a `Ball` due membri `speed` ed `angle`, e inizializzarli ai valori di default. - Aggiungere a `Ball` un metodo `move()`, e a `State` un metodo `update()`, che prendono in input un `float elapsed`. Nel main loop viene chiamato `State.update()`, che a sua volta richiama `Ball.move()`, che a sua volta modifica incrementalmente `Ball.pos`.

Parte 2.b - Start & Pause!

Obiettivo:

Permettere all'utente di far partire il gioco in modo controllato, di metterlo in pausa e di riprenderlo. In aggiunta, se la finestra perde il focus, il gioco va automaticamente in pausa.

Sviluppo:

- Aggiungere a `State` un campo `bool pause`, inizializzato a `true` (quando il gioco parte, è fermo). - Aggiungere a `window.handleEvents` la registrazione della seguente lambda expression: `[&state] (const auto& event) { handle(event, state)`. L'uso di `const auto& event` come parametro permette di effettuare la registrazione una volta sola per tutte le callback specifiche per la gestione dei singoli eventi a cui bisognerà reagire modificando lo stato del programma. La funzione corretta per l'evento giusto viene chiamata tramite overloading. - Per far funzionare questo meccanismo, fra le handle bisogna inserirne una generica, vuota, `template <typename T> void handle (T& event, State& state) {}` per catturare tutti gli eventi che non c'è bisogno di gestire. - A questo punto si può aggiungere una handle che gestisce l'evento

`sf::Event::KeyPressed` e prende come parametro anche lo stato. Questa funzione inverte il campo booleano `pause` di `State` se è stata premuta la barra spaziatrice (`sf::Keyboard::Scancode::Space`). - Aggiungere anche una handle per l'evento `FocusLost` che prende in input `state`, e pone `pause` a `true`. - Modificare `state.update()` per leggere il campo `pause`, se è a `true`, `update()` deve lasciare tutto fermo.

Parte 2.c: - Racchetta

Obiettivo:

Il giocatore deve poter muovere la racchetta a destra e a sinistra usando le frecce sulla tastiera.

Sviluppo:

- Aggiungere la handle per trattare l'evento `KeyReleased`, e poi catturare gli scancode per la freccia destra e sinistra della tastiera sia in `KeyReleased` che in `KeyPressed`. L'aggiunta dell'evento `KeyReleased` permette un controllo preciso della racchetta, che sarebbe altrimenti impossibile se ci limitassimo ad utilizzare le ripetizioni dell'evento `KeyPressed`, che sono dipendenti dalla configurazione di sistema del key repeat nel sistema operativo. - Aggiungere a `State` due membri Booleani `bool move_paddle_left` e `bool move_paddle_right`, inizializzati a `false` nel costruttore, messi a `true` quando vengono premute le rispettive frecce sulla tastiera, e di nuovo a `false` quando vengono rilasciate. - Aggiungere a `Paddle` un membro `float speed`, inizializzato da una variabile globale `float paddle_initial_speed` nel costruttore, e due nuovi metodi `move_left()` e `move_right()`. - `Paddle::move_left` e `Paddle::move_right()` funzionano in modo analogo a `Ball::move()`, semplicemente facendo un displacement (rispettivamente negativo e positivo) della coordinata `pos.x` della racchetta, scalato sul tempo trascorso fra il frame corrente e il frame precedente. Questo ci permette di definire in modo preciso la velocità della racchetta in pixel al secondo, come per la pallina.

Capitolo 3 - Collisioni e rimbalzi

I due elementi inseriti fino ad ora - pallina e racchetta - devono interagire correttamente con il campo di gioco e fra di loro. La pallina deve rimbalzare sui bordi laterali e superiore, deve rimbalzare contro la racchetta, e deve cadere giù dal bordo inferiore se viene mancata dalla racchetta.

Parte 3.a - Bordi del campo

Obiettivo:

Pallina e racchetta devono muoversi solo entro i limiti del campo. La pallina deve rimbalzare contro tutti i bordi, e la racchetta deve fermarsi contro i bordi sinistro e destro.

Come idea generale, Per la

Sviluppo:

- Aggiungere a **State** due metodi: **field_limits()** che gestisce le collisioni di racchetta e pallina coi limiti del campo, e **collisions()** che gestisce le collisioni in generale. Per il momento, **collisions()** viene chiamato alla fine di **update()** e si limita a chiamare **field_limits()**. - Il metodo **field_limits()** controlla se la posizione attuale di **paddle.pos** e **ball.pos** pone questi elementi fuori dai bordi e nel caso reagisce di conseguenza. - Se la racchetta supera il limite del campo a sinistra, **paddle.pos.x** viene forzata a zero, e se lo supera a destra, **paddle.pos.y** viene forzata alla larghezza della finestra meno la dimensione della racchetta. - Se la pallina supera il limite del campo a sinistra o destra, si inverte la componente orizzontale del suo movimento, e se lo supera in alto o in basso, si inverte la componente verticale. - Per gestire le inversioni conviene creare due funzioni **reflect_horizontal** e **reflect_vertical** che prendono in input un **sf::Angle** e ne restituiscono un altro in cui sia stata invertita rispettivamente la componente orizzontale o quella verticale della direzione. In pratica, il modo più naturale di farlo è passare dalla rappresentazione in coordinate polari a quella in coordinate cartesiane, cambiare segno a x o y, e poi tornare in coordinate polari. La classe **sf::Vector2f** fornisce sia il costruttore per modulo e angolo già usato in precedenza, che un metodo **angle()** che restituisce l'angolo a partire dal vettore. Notare che il valore del modulo in questo caso è ininfluente e può essere semplicemente 1.0, l'obiettivo al momento è soltanto generare un nuovo angolo con una delle due componenti di direzione invertita per simulare un rimbalzo con riflessione.

Problemi ed esperimenti:

- provare a sperimentare diversi valori di velocità e framerate. In certi casi, si potrebbe incorrere in un bug piuttosto fastidioso. Se il framerate è basso e la pallina si muove lentamente, potrebbe essere ancora parzialmente fuori del bordo anche nel frame successivo a un'inversione di direzione, e rimanere "intrappolata" lungo il bordo da una sequenza ripetuta di falsi rimbalzi. - Se non si riesce a riprodurre il bug, si può provare ad alzare il limite del bordo inferiore sopra la posizione di partenza della pallina (due o tre volte la dimensione della racchetta, ad esempio) e osservare cosa succede.

Parte 3.b - Collisioni robuste sui bordi

Obiettivo:

Evitare falsi rimbalzi dentro ai bordi.

Sviluppo:

- L'idea generale è di far rimbalzare la pallina solo se si sta effettivamente muovendo verso un bordo, e di non fare nulla se invece si sta già allontanando. - Una possibilità è di aggiungere un'ulteriore condizione ai controlli dentro **field_limits()**. L'inversione di direzione va fatta quando la posizione della pallina la pone parzialmente fuori dal bordo e la sua direzione attuale la spinge ulteriormente fuori. In questo modo, se la pallina risulta ancora dentro al bordo ma si sta allontanando da esso, può continuare ad allontanarsi indisturbata.

- Per far questo, conviene scomporre l'angolo di direzione della pallina nelle sue componenti `x` e `y`, con una chiamata al solito costruttore per coordinate polari, già dentro `field_limits()`. - A quel punto, per ogni bordo orizzontale o verticale si può aggiungere un controllo sul segno della componente `x` o `y`.

Parte 3.c - Collisione e rimbalzo fra pallina e racchetta

Obiettivo:

controllare se la pallina colpisce la racchetta e nel caso farla rimbalzare. Se la pallina manca la racchetta, il gioco viene riavviato.

Sviluppo:

- Aggiungere a `Paddle` un metodo `hit()` che prende in input una `Ball` e restituisce `true` se questa colpisce la racchetta. Questo metodo calcola la direzione di movimento della pallina e controlla che la direzione punti verso il basso e che la pallina intersechi i limiti della racchetta. In pratica, in verticale è sufficiente confrontare che la `y` della pallina abbia superato la quota di intersezione col fondo, e in orizzontale è necessario fare il controllo rispetto ai limiti destro e sinistro della racchetta. - Modificare il metodo `collisions()` in modo che, dopo aver chiamato `field_limits()`, controlli se c'è una collisione con la racchetta, attraverso il metodo `hit()` di `Paddle` e, in tal caso, faccia rimbalzare la pallina verticalmente. - Aggiungere a `State` un metodo `restart()` che riporta il gioco alla situazione iniziale: pallina e racchetta al centro, gioco in pausa. - Modificare il metodo `field_limits` in modo che quando la pallina incontra il limite inferiore della finestra non ci siano rimbalzi, ma venga chiamato il metodo `restart`. Notare che il riavvio del gioco deve avvenire solo dopo che la palla è scomparsa fuori dalla finestra.

Problema:

Il gioco minimale, nel suo stato attuale, è fisicamente corretto, ma è piuttosto noioso. L'angolo di riflessione si inverte, ma non cambia mai. Se non avete mai giocato ad arkanoid o a un gioco similare, è il momento di provare (ne trovate innumerevoli cloni e versioni online, incluso un easter egg di google), provatelo e notate come funziona l'interazione fra pallina e racchetta.

Parte 3.d - cambio di direzione della pallina in base alla collisione sulla racchetta

Obiettivo:

L'angolo di rimbalzo della pallina sulla racchetta deve ignorare l'angolo di direzione precedente e dipendere solo dalla posizione in cui la pallina colpisce la racchetta. In particolare, una collisione esattamente sul centro dovrebbe far rimbalzare la pallina esattamente in verticale, e più il colpo è lontano dal centro, più l'angolo di direzione di rimbalzo dovrebbe avvicinarsi ad una direzione vicina all'orizzontale.

Sviluppo:

- Aggiungere una costante globale `sf::Angle left_max_angle` per fissare un

limite all'angolo di rimbalzo. Questo valore deve essere impostato in modo che la pallina torni sempre su verso l'alto, anche se di poco. - Aggiungere a **Paddle** un metodo `void strike(Ball&)` per la gestione del rimbalzo. Sostituire in **State::collisions** le istruzioni di gestione della collisione con una chiamata a questo metodo. - Il metodo **strike** prima controlla se c'è una collisione (**hit**) con la pallina; in caso affermativo, calcola dapprima la distanza orizzontale tra i centri di pallina e racchetta e lo utilizza per calcolare l'angolo di uscita dopo il rimbalzo: questo sarà ottenuto calcolando l'interpolazione lineare tra **left_max_angle** e **right_max_angle**, quest'ultimo calcolato riflettendo in orizzontale **left_max_angle**. Per il calcolo dell'interpolazione lineare implementiamo una funzione helper **linear_interpolation** che prende tre valori: **v0** e **v1** che rappresentano i limiti dell'interpolazione e **t** compreso in $[0,1]$ che è il parametro di interpolazione; per $t=0$ restituiamo **v0**, per $t=1$ restituiamo **v1** e per valori intermedi restituiamo un valore che varia linearmente con **t** tra tali estremi. Il valore corretto di **t** da passare alla funzione viene calcolato come distanza tra i centri di pallina e racchetta, normalizzata sull'ampiezza della racchetta più quella della pallina, e aumentata di 0.5 per ricondurla all'intervallo $[0,1]$. L'angolo calcolato per interpolazione lineare viene utilizzato per aggiornare l'angolo di movimento della pallina. - Notare che conviene passare alla funzione di interpolazione lineare un valore in radianti. SFML fornisce il metodo **asRadians()** e la funzione **sf::radians()** per convertire facilmente un **sf::Angle** in radianti e viceversa.

Problema:

C'è un bug che potrebbe essere difficile da notare. Per riprodurlo, bisogna provare a colpire la pallina con uno dei bordi verticali della racchetta invece che con la parte superiore. Per rendere la cosa più facile da far accadere, si può provare ad ingrandire l'altezza della racchetta.

Parte 3.e - gestire i bordi della racchetta

Obiettivo:

quando il rimbalzo avviene sui bordi verticali della racchetta, la palla deve rimbalzare verso l'alto solo se la pallina colpisce il bordo verticale nella metà superiore (salvataggio riuscito). Se colpisce la metà inferiore, la palla dovrebbe rimbalzare verso il basso e andare fuori dal campo (salvataggio fallito).

Sviluppo:

basta modificare la conclusione del metodo **Paddle::strike()**. Appena prima di impostare **ball.angle** al nuovo angolo, bisogna computare la coordinata **y** del centro della pallina e la coordinata **y** del centro della racchetta. Se la prima è maggiore della seconda, il salvataggio è fallito e bisogna riflettere verticalmente il nuovo angolo calcolato in precedenza prima di assegnarlo alla pallina.

Calibrazione:

Ora che il movimento e l'interazione di pallina e racchetta sono completi, bisognerebbe calibrare la velocità dei due elementi in modo che il gioco sia

responsivo. La pallina dovrebbe andare più veloce o più lenta della racchetta?

Capitolo 4 - Il muro di mattoni

Aggiunta al gioco dell'elemento finale: il muro di mattoni, che verrà progressivamente distrutto dalle collisioni con la pallina. Il muro, come in Breakout, è una griglia di rettangoli posta nella parte superiore del campo da gioco. Fra il muro e i bordi del campo di gioco rimangono dei corridoi, in cui la pallina può riuscire ad entrare se opportunamente lanciata.

Parte 4.a - struttura e disegno del muro

Obiettivo:

Creare una struttura adeguata per rappresentare per muro e mattoni, e disegnare questi elementi a schermo.

Struttura di base:

- Definire una `struct Block` per rappresentare un singolo mattone e una `struct Wall` per rappresentare l'intero muro. - `Block` ha due campi `sf::Vector2f pos` e `sf::Vector2f size` per memorizzare posizione e dimensioni del mattone, e un costruttore esplicito che riceve in input posizione e dimensione e inizializza i due campi. - `Wall` ha un solo campo `std::vector<Block> blocks` per mantenere il muro, e un costruttore di default senza parametri.

Nuove costanti globali: - `sf::Vector2f wall_displacement` configura la distanza in pixel del muro dai bordi. `y` tiene la distanza del muro dal bordo superiore, `x` la distanza del muro dal bordo sinistro e destro. - `sf::Vector2f wall_size` per la dimensione del muro in altezza e larghezza. - `sf::Vector2i block_num` quanti mattoni sono presenti nel muro in altezza e larghezza. - `sf::Color block_fill_color` per il colore interno di un mattone. - `sf::Color block_outline_color` per il colore di bordo di un mattone. - `float block_outline_thickness` per lo spessore della linea di bordo.

Sviluppo: - Il costruttore di `Wall` deve calcolare la dimensione del singolo mattone in base alle costanti di programma (area occupata dal muro e numero di mattoni nelle due dimensioni), quindi istanziare i mattoni di tale dimensione nelle posizioni opportune, ciclando su righe e colonne, e inserendoli nel vector. - Aggiungere un metodo `draw()` a `Wall` che cicla su tutti i mattoni, chiamando per ognuno il suo metodo `Block::draw()`. - Aggiungere un metodo `draw()` a `Block` che disegna un singolo mattone come `RectangleShape` alla sua posizione e della sua size, e con i colori e spessori definiti nelle costanti globali. - Aggiungere un oggetto `Wall wall` ai campi di `State`, e chiamare `wall.draw()` dentro `State::draw()`.

Parte 4.b - collisioni fra pallina e muro

Obiettivo:

rendere il muro interattivo. La pallina deve rimbalzare contro i mattoni, e ogni volta che un mattone viene colpito, scompare.

Sviluppo: - Aggiungere un campo `bool intact` a `Block`, inizializzato a `true` nel costruttore, per indicare se il mattone è intatto o è stato distrutto. - Aggiungere a `Block` il metodo `bool is_inside (sf::Vector2f)`, che prese le coordinate di un punto, ci dice se questo è all'interno del mattone. Basta confrontare le coordinate del punto con i limiti del rettangolo corrispondente al mattone. - Aggiungere a `Block` il metodo `void hit(Ball&)`. Il metodo controlla se la pallina ha colpito il mattone e, in caso di collisione, fa rimbalzare la pallina e distrugge il mattone. Un'implementazione corretta di questo metodo non è banale: bisogna capire non solo se la pallina è in collisione con il mattone, ma anche da quale lato lo sta colpendo, per poter capire come farla rimbalzare. Un'implementazione approssimata potrebbe prima calcolare gli estremi inferiore, superiore, destro e sinistro in cui il cerchio della pallina tocca il quadrato in cui è inscritta. Per ciascuno di questi punti si può vedere se è all'interno del mattone e se la direzione della pallina la porterebbe a proseguire dentro il mattone da quel lato. In caso affermativo, bisogna riflettere la direzione della pallina e distruggere il mattone. - Aggiungere anche a `Wall` un metodo `void hit(Ball&)`. Per un'implementazione semplice basta effettuare un loop su tutti i mattoni del muro, e per quelli ancora intatti verificare le collisioni con la pallina chiamando `block.hit (ball)`.

Conclusioni

Tutti gli elementi fondamentali di Breakout sono stati sviluppati. Potrebbe ancora essere necessario calibrare adeguatamente tutte le costanti del gioco (dimensione degli elementi, numero dei mattoni, velocità di pallina e racchetta).

Il gioco, allo stato attuale, ha ancora alcune limitazioni, e potrebbe essere esteso in molti modi.

Problemi e limitazioni

Limiti della collisione fra pallina e mattoni - 1:

- Un ciclo su tutti i mattoni per ogni frame è accettabile, in quanto il numero di mattoni è relativamente piccolo e la macchina molto veloce. Tuttavia, si tratta di un metodo di collision detection molto inefficiente. Nel caso specifico, poiché il muro ha una struttura a griglia regolare, conoscendo la posizione e le dimensioni dei mattoni (come costruiti all'inizio) sarebbe possibile verificare se siano intatti i soli mattoni la cui area si sovrappone a quella della pallina, ignorando tutti gli altri. Per scene più complicate è invece necessario ricorrere a strutture dati che possano restringere in campo degli oggetti con in quali si hanno potenziali collisioni. La collision detection rappresenta un'area importante del calcolo geometrico per la grafica ed è un argomento molto esteso.

Limiti della collisione fra pallina e mattoni - 2:

- La pallina è tonda, ma le sue collisioni sono semplificate, e sfruttano la bounding box quadrata per le collisioni con bordi del campo e racchetta, e un quadrato inscritto ruotato di 90 gradi per le collisioni coi mattoni. Questo rende la

computazione delle collisioni veloce da calcolare e semplice da scrivere. Questa semplificazione non pare essere un problema per il campo, ma facendo molta attenzione si può notare che, per quando riguarda i mattoni, è effettivamente un'approssimazione imprecisa. Sono possibili molte alternative più precise. Per ispirazioni e suggerimenti, si può consultare questa pagina su [Stack Overflow](#). Va notato che una buona soluzione deve risolvere contemporaneamente due problemi: non solo computare una collisione cerchio - rettangolo precisa, ma anche computare la nuova direzione della pallina con riflessione dell'angolo, in base a dove la pallina ha colpito e alla direzione precedente. Il fatto che i mattoni siano tutti rettangoli allineati agli assi è una grande semplificazione, altrimenti il problema sarebbe ancora più complesso.

Estensioni e varianti

Sviluppo del gioco Pong:

- Per una facile variante, si può togliere il muro di mattoni, aggiungere una seconda racchetta dal lato opposto del campo e gestire l'uscita della pallina anche da quel lato per implementare il gioco Pong [Atari, 1972] a due giocatori. La seconda racchetta può utilizzare la stessa classe della prima per il disegno, mentre richiede qualche modifica per le regole di rimbalzo. Dovranno essere utilizzati altri tasti per il suo controllo nella GUI. Nota: nel gioco originale le racchette erano disposte sui lati sinistro e destro e si muovevano verticalmente. Non è difficile adattare il programma a questo scenario, ma richiede qualche cambiamento.

Miglioramenti visuali:

- Utilizzo di texture anche sui mattoni. - Eventuale colorazione dipendente dalla posizione del mattone. - Eventuale animazione della rottura di un mattone tramite *sprite*.

Vite, punteggi e livelli:

- Gestione del punteggio e delle vite (con visualizzazione dinamica delle variazioni utilizzando `sf::Text`). - Variazione della velocità e/o dimensione della pallina al variare del punteggio. - Gestione dei livelli, tramite una ristrutturazione del codice (livelli diversi possono richiedere dimensioni dei mattoni e numeri diversi, velocità e dimensioni della pallina diverse, etc.). Può essere conveniente creare una struttura dati che memorizza tutti i parametri che caratterizzano un livello e mantenere un array di tali strutture numerato sui livelli.

Estensioni al gameplay:

- Aggiunta di ostacoli fissi e/o di mattoni di diversi tipi (ad esempio alcuni mattoni potrebbero aver bisogno di più colpi per essere distrutti). - Agenti autonomi dinamici, come oggetti che si muovono dal muro verso il basso e devono essere intercettati o evitati dalla racchetta (powerups, o handicap). - Effetti che aumentano o diminuiscono dinamicamente certi parametri di gioco, come velocità di pallina e racchetta, dimensioni della racchetta.

Controllo della racchetta tramite mouse:

- Si può sperimentare un input da mouse invece che da tastiera. Uno dei tasti del mouse mette in pausa, e i movimenti orizzontali del mouse muovono la racchetta in orizzontale. Notare che la velocità della racchetta non sarebbe più costante, ma dipenderebbe dalla velocità con cui si muove il mouse. Dovrebbe muoversi alla velocità massima del mouse, o dovrebbe essere limitata in qualche modo, e nel caso come?

Resize e fullscreen:

- L'attuale funzione di resize non è particolarmente adatta a questo gioco, essendo che il campo è di dimensioni fisse. Se si restringe la finestra, parte del campo viene nascosta, mentre se viene ingrandita, non si aggiunge niente e il gioco non viene neppure mostrato più grande. Come si può migliorare questa parte, ed eventualmente implementare una modalità fullscree?

Sonoro:

- Aggiungere effetti sonori o della musica di sottofondo sfruttando la [sound API](#) di SFML.

... E molto altro ...